



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

软件工程实验报告

智能城市生成系统需求分析报告

林晖鹏

年级：2023 级

专业：计算机科学与技术

指导教师：李起成

2026 年 4 月 16 日

摘要

智能城市生成系统面向城市市场快速构建、三维方案调整、动态效果展示与智能交互辅助等应用需求,目标是在 Blender 插件执行环境的基础上,结合程序化建模和大语言模型技术,形成一个支持多角色协同使用的智能化场景生成平台。本文档参考软件需求规格说明 (Software Requirements Specification, SRS) 的组织思想,对系统的业务目标、使用对象、数据范围、核心功能、质量要求和运行环境进行分析与规约。

报告在章节组织上遵循课程作业要求,整体分为引言、任务概述、业务描述、数据需求、功能需求、性能/非功能需求和系统运行要求七个部分。其中,引言部分交代文档目的、项目背景与术语约定;任务概述部分说明系统目标、产品定位、用户特点及假定约束;业务描述部分围绕系统总流程与关键子流程展开;数据需求部分对核心数据对象、数据流和数据字典进行说明;功能需求部分按照模块划分方式给出详细需求;性能与非功能需求部分从准确性、及时性、可扩展性、可靠性、安全性与可维护性等角度提出质量要求;系统运行要求部分则从软硬件环境与接口协同方式上对系统运行条件进行约束。

本报告特别突出一个创新功能,即接入大语言模型实现多模态指令解析,并根据解析结果自动调用 Blender 插件能力完成场景建模或场景修改任务。相较于传统依赖纯手工建模的方式,该系统试图将文本、草图、点线集和参数表单等多种输入形式统一转化为结构化任务,以提高建模效率、降低专业门槛,并增强用户与系统之间的交互自然性。本文档形成的需求分析结果,可作为后续原型设计、概要设计和系统实现的重要依据。

关键词: 智能城市生成系统; 需求分析; SRS; Blender; 多模态交互; 大语言模型

目录

一、 引言	3
(一) 编写目的	3
(二) 项目背景	3
(三) 术语说明	4
二、 任务概述	4
(一) 任务目标	4
(二) 用户特点	5
(三) 假定与约束	5
三、 业务描述	7
(一) 系统总业务流程图及其描述	7
(二) 各个子业务流程图及其描述	7
四、 数据需求	11
(一) 数据需求描述	11
(二) 数据流程图	13
(三) 数据字典	13
五、 功能需求	15
(一) 功能划分	15
(二) 功能描述	15
(三) 功能补充说明	21
(四) 关键功能场景说明	21

六、 性能/非功能需求	22
(一) 系统的准确性	22
(二) 模型解析的可解释性与容错性	22
(三) 系统的及时性	22
(四) 系统的可扩充性	23
(五) 系统的易用性	23
(六) 系统的易维护性	23
(七) 系统的标准性	23
(八) 系统的先进性	23
(九) 系统的安全性	23
(十) 系统的可靠性与可移植性	24
七、 系统运行要求	25
(一) 硬件配置要求	25
(二) 软件配置要求	25
(三) 运行方式要求	26
(四) 部署与维护要求	26
(五) 外部接口要求	26
(六) 运行要求补充说明	26
八、 总结	26

一、 引言

(一) 编写目的

为了对智能城市生成系统进行较为系统、完整和规范的需求分析，并形成一份能够为后续设计、开发、测试和汇报提供依据的文档。按照软件需求规格说明文档的基本思想，需求文档不应只是一份功能清单，而应明确软件在什么条件下需要完成哪些工作、输入和输出之间如何转换、系统需要满足哪些质量要求、以及系统在现实环境中受到哪些限制。只有当这些问题在需求阶段被说清楚，后续课程大作业的软件设计与实现工作才具有稳定基础。

对于本次课程大作业而言，需求分析报告还有两层实际意义。其一，它是整个项目讨论和分工的统一参照，能够减少因理解偏差导致的返工；其二，它也是后续评审、测试和修改的重要依据。若需求文档写得过于空泛，则后续实现过程很容易陷入“边做边猜”的状态；若需求文档写得过于随意，则无法体现软件工程方法中“先理解问题、再抽象需求、最后组织实现”的基本过程。因此，本报告既服务于作业提交，也服务于完整的软件工程训练过程。

(二) 项目背景

近年来，智慧城市、数字孪生、三维可视化、虚拟仿真和程序化内容生成等技术持续发展，城市空间的数字化表达需求日益增强。在城市规划展示、园区方案设计、游戏场景搭建、教学演示和行业仿真分析等场景中，构建可观察、可编辑、可重复生成的三维城市场景已成为较为常见的需求。然而，传统三维建模方式通常依赖人工逐步搭建，存在建模效率低、修改成本高、风格一致性难以保持、专业门槛较高等问题。

Blender 作为开源三维内容创作平台，拥有较成熟的插件扩展机制和脚本控制能力，适合作为城市场景执行与编辑的底层环境。通过 Blender 插件，可将道路生成、建筑布设、绿化配置、生态对象添加和资产替换等能力封装为可调用的功能组件，从而使上层主系统能够围绕任务组织和交互控制开展工作。与此同时，大语言模型在自然语言理解、指令拆解、参数补全和工具调用方面表现出较强能力，这为“让用户通过自然语言参与三维场景生成”提供了现实可能。

基于上述背景，本项目拟构建一套智能城市生成系统。该系统不只是一个 Blender 插件，也不只是一个展示型前端界面，而是一个由主控系统、Blender 插件执行端、数据与资源管理模块、仿真模块和大语言模型服务共同组成的协同平台。它面向的核心问题是：如何以更低的表达成本、更高的生成效率和更好的修改灵活性，帮助不同类型用户完成城市场景的创建、调整、分析和展示。

(三) 术语说明

为避免后文表述出现歧义，现对本文涉及的主要术语进行统一说明。

表 1: 术语说明表

术语	含义说明
智能城市生成系统	本文分析的目标软件系统，用于城市三维场景生成、编辑、仿真与管理
主系统	面向用户的控制平台，负责交互、任务组织、权限控制和结果展示
Blender 插件	在 Blender 环境中执行具体场景生成、对象修改和资源替换任务的插件组件
资产	用于构建场景的资源对象，包括道路、建筑、树木、庭院、水体装饰等 2D/3D 元素
布局点线集	由控制点、控制线、边界和区域构成的几何布局输入数据
多模态指令	同时包含文本、草图、参数表单或其他输入形式的复合型用户指令
动态仿真	对车辆、人群、船只等动态对象进行规则驱动的可视化演示过程
LLM	Large Language Model，即大语言模型，用于自然语言理解与任务解析
SRS	Software Requirements Specification，即软件需求规格说明文档

二、 任务概述

(一) 任务目标

智能城市生成系统的建设目标应覆盖多角色登录、主系统与 Blender 插件交互、场景生成、布局与生态控制、智能交互与动态仿真等方面。结合本项目定位，本小节将总体任务目标概括为以下几点：

1. 建立一个能够支持城市场景快速创建与持续编辑的生成平台，使场景建模师能够基于模板、参数与资产库快速生成城市道路、建筑、植被和庭院等基础内容。
 2. 建立一个支持多输入方式的任务表达机制，使用户既可以通过参数表单进行精确配置，也可以通过点线集、草图甚至自然语言表达场景修改意图。
 3. 建立主系统与 Blender 插件的协同执行机制，使上层任务组织和底层三维执行之间具有清晰的接口边界和稳定的任务回传机制。
 4. 建立动态仿真与结果展示能力，使生成出来的场景不仅能够静态查看，还能够通过车辆、人群和船只等对象的动态表现支持方案展示和分析。
 5. 建立多角色权限和日志治理能力，使系统管理员能够对用户、角色、资源、模型服务和操作日志进行管理，从而保证系统具备持续运行条件。
 6. 建立基于大语言模型的创新能力，将自然语言与多模态输入解析为结构化任务，并自动调用插件完成部分场景建模操作。
- 上述目标主要用于界定系统建设范围和核心能力边界。

1. 智能交互能力边界说明

由于本系统引入了大语言模型服务，因此有必要在需求阶段提前界定其职责边界，以避免把模型能力与底层执行能力混为一谈。从系统分工来看，LLM 的主要职责是理解用户输入、补全语义上下文、识别对象及其关系、生成结构化任务建议，并为主系统提供可供确认的解释结果；

而 Blender 插件的主要职责则是依据结构化任务完成几何生成、对象替换、布局调整和场景文件更新。换言之，LLM 负责“理解用户想做什么”，插件负责“把任务真正执行出来”。

之所以需要强调这一边界，是因为三维场景生成并非单纯的文本生成问题。即使模型能够从自然语言中提取出“区域”“道路”“湖泊”“船只”等语义要素，它仍然需要依赖当前场景上下文、资产库状态、插件能力边界和参数约束条件，才能形成可执行任务。因此，在需求分析中不应把 LLM 描述为一个可以直接替代建模逻辑的“万能模块”，而应将其视为连接用户意图与系统执行链路的中间解释层。只有这样，后续功能划分、接口设计和错误回退机制才能保持清晰。

从用户视角看，引入 LLM 的根本意义在于降低表达门槛，而不是取消确认环节。系统应允许用户使用更自然的方式表达需求，但仍应在关键任务执行前向用户展示解析后的结构化结果，并在存在歧义、上下文不足或约束冲突时提示补充信息或进行二次确认。这样既能发挥模型对复杂输入的理解优势，也能避免将不确定性直接传递到最终场景结果中。

(二) 用户特点

系统的目标用户包括**场景建模师**、**行业分析师**和**系统管理员**三类。不同用户对系统的期待和需求不同，因此需求组织上不能把所有角色混合为一个“普通用户”。

表 2: 用户特点分析表

用户角色	核心职责	主要关注点	典型使用特点
场景建模师	创建、调整和导出三维城市场景	生成效率、参数控制能力、局部替换灵活性、可视化效果	使用频率最高，熟悉建模概念，但希望减少重复劳动
行业分析师	观察方案差异、查看动态结果、辅助完成展示和论证	结果清晰度、方案可比性、操作简单性、导出便捷性	更关注结果展示和解释，不希望承担复杂建模操作
系统管理员	维护用户、角色、权限、资源、日志与模型配置	稳定性、安全性、可追踪性、配置统一性	主要在后台工作，关注系统治理而非场景生成本身

场景建模师是本系统的主导型用户。他们通常具备一定的三维场景理解能力，能够判断道路结构、建筑分布和植被搭配是否合理，但并不意味着他们愿意重复进行大量机械操作。因此，对于建模师而言，系统最重要的价值是提高生成效率并降低修改成本，而不是单纯提供一个“能编辑”的界面。

行业分析师则更偏向结果型使用。他们通常更关注“方案最后呈现成什么样”“不同布局产生了什么差异”“动态效果是否符合展示目标”，因此其使用路径应尽量短、结果展示应尽量清晰。这类用户未必需要深度操作 Blender 插件，也未必愿意理解建模参数的全部细节。

系统管理员承担的是系统治理职责。其关注点在于账号是否规范、权限是否隔离、资源库和模型服务是否可控、错误和操作是否可追踪。若系统缺少这一层能力，那么即便前台功能丰富，也难以支撑真实软件系统的长期维护。

(三) 假定与约束

结合课程大作业的需求和场景，这里将假定与约束归纳如下。

1. 基本假定

1. 假定系统以 Blender 作为三维场景执行环境，主系统提交的核心建模任务最终均由 Blender 插件执行。
2. 假定系统具有一定规模的基础资产库，包括道路、建筑、树木、庭院、水体、山体装饰以及动态对象资源。
3. 假定系统可以接入可用的大语言模型服务，用于自然语言指令解析和任务结构化输出。
4. 假定课程作业重心在需求分析与系统构思，因此部分指标采用合理估计方式给出，而非工业级基准测试结论。

2. 主要约束

1. 系统生成质量受到资产质量、草图清晰度、参数设置精度和模型解析能力的共同影响，因此对自动生成结果不能作绝对正确的承诺。
2. 受硬件性能与插件执行速度限制，复杂场景的生成和仿真可能需要异步处理，因此系统应允许任务排队、进度回传与失败提示机制存在。
3. 草图识别仅能作为辅助输入手段，自动提取结果应允许人工修订，以避免识别误差直接影响最终布局。
4. 动态仿真的主要定位是展示与辅助分析，而非交通工程级或物理级高精度仿真系统，因此需求描述应聚焦可视化效果和方案比较价值。
5. 系统的需求表述应尽量使用清晰、可测试、可理解的语言，避免仅用“智能”“友好”“高效”等不可验证词汇作为核心表述。

三、 业务描述

(一) 系统总业务流程图及其描述

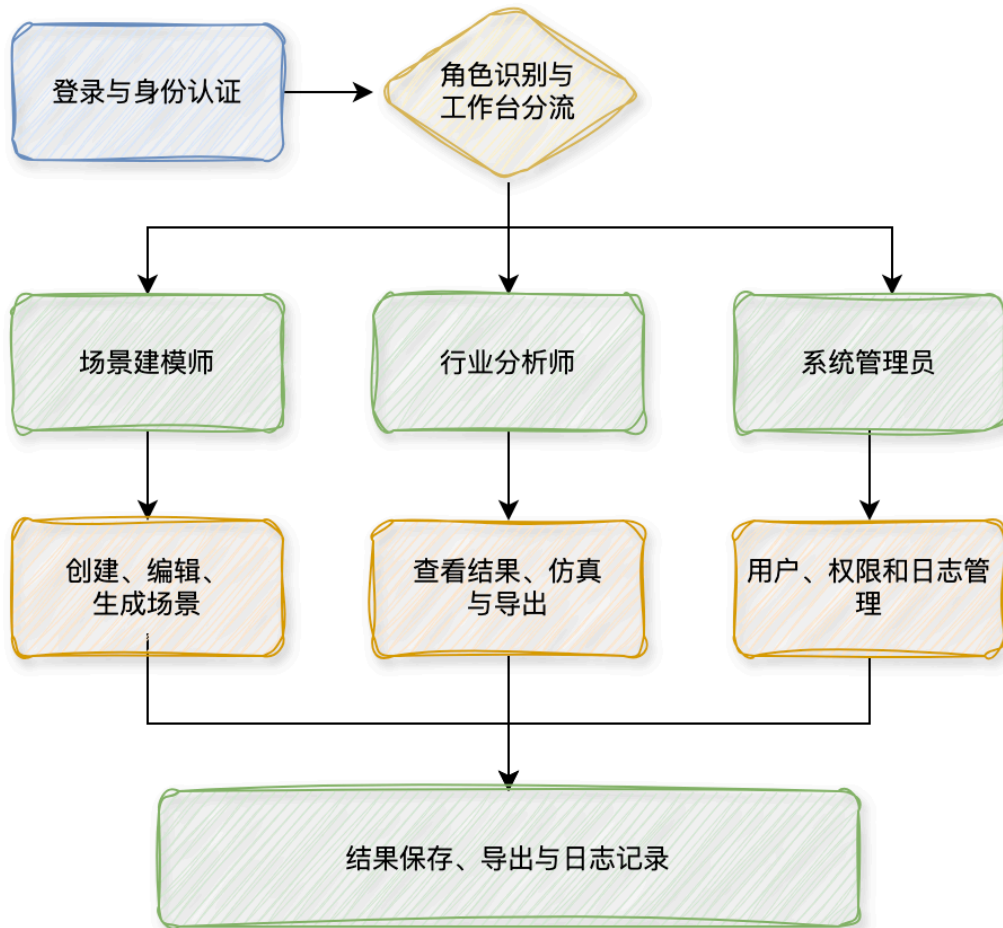


图 1: 系统总业务流程图

系统总业务流程从统一登录入口开始，系统首先完成身份认证，并根据角色将用户分流到不同工作台。场景建模师进入建模工作台后，主要完成场景创建、布局调整、插件调用和结果修订；行业分析师进入分析工作台后，主要完成场景查看、动态仿真、方案比较与结果导出；系统管理员进入后台管理工作台后，主要完成用户管理、权限配置、模型服务维护和日志审计。不同角色虽然业务重点不同，但最终都将形成可保存、可导出、可追踪的结果或记录。

智能城市生成系统并不是单一用户使用的建模工具，而是一个面向三类角色协同使用的软件系统。它既要支持建模师完成核心生成任务，也要支持分析师完成结果消费和方案比较，还要支持管理员完成治理与维护。因此，系统业务分析不能只围绕“如何生成场景”展开，还应体现角色分流、结果使用和后台管理等完整软件行为。

(二) 各个子业务流程图及其描述

为了更完整地刻画系统业务过程，本文在子业务流程层面分别从三类角色的使用路径出发进行分析，并进一步对场景建模师最核心的三类操作流程进行细化描述。这样组织的原因在于：

一方面，题目已经明确给出三类用户，因此角色流程应形成闭环；另一方面，场景建模师承担系统最复杂、最核心的业务活动，需要进一步拆分才能体现需求分析的充分性。

3.2.1 场景建模师业务流程

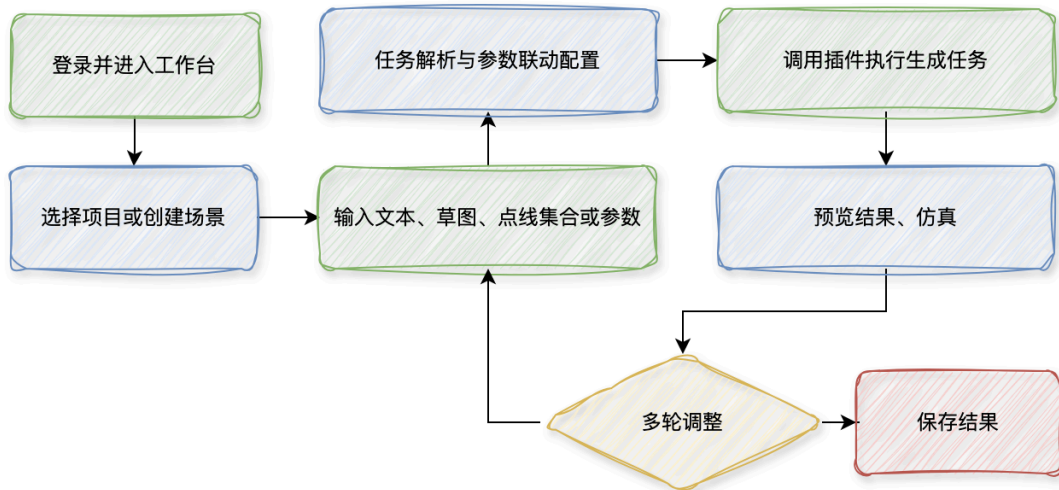


图 2: 场景建模师业务流程图

场景建模师是系统最核心的业务用户，其业务流程体现了系统“从设计意图到场景结果”的主线。建模师登录后进入建模工作台，选择已有项目或新建场景，随后通过文本指令、草图、点线集或参数表单表达自己的设计意图。主系统对这些输入进行解析，并联动展示相关参数选项；当用户确认后，系统调用 Blender 插件执行场景生成、对象替换或布局调整；结果返回后，建模师可结合动态仿真结果进一步修改，最后完成项目保存和场景导出。该流程覆盖了系统最主要的使用场景，也是后续细化流程的基础。

3.2.2 行业分析师业务流程

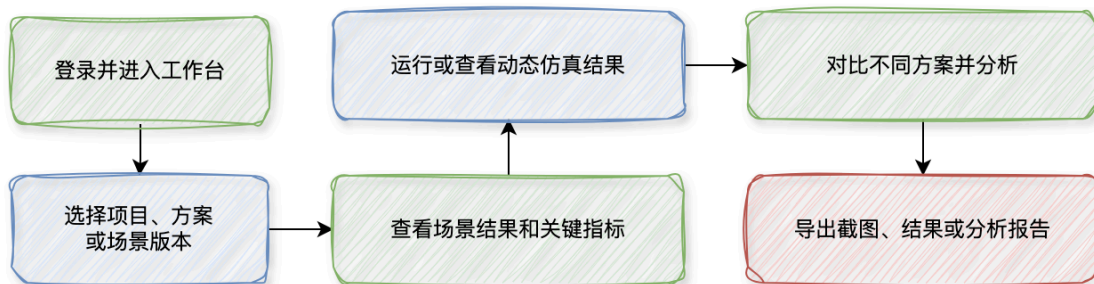


图 3: 行业分析师业务流程图

行业分析师的业务重点不在于构建场景本身，而在于利用系统生成的场景结果完成观察、比较和分析工作。因此，这类用户登录后首先进入分析工作台，并选择待查看的项目、方案或场景版本；在此基础上，系统向其展示场景结果、仿真结果和相关可视化信息；分析师可以对不同布局方案、不同生态配置或不同动态效果进行横向比较，并形成分析意见；最后，系统支持其导出截图、结果文件或阶段性分析报告。该流程强调的是结果消费能力，而不是建模过程能力。

3.2.3 系统管理员业务流程

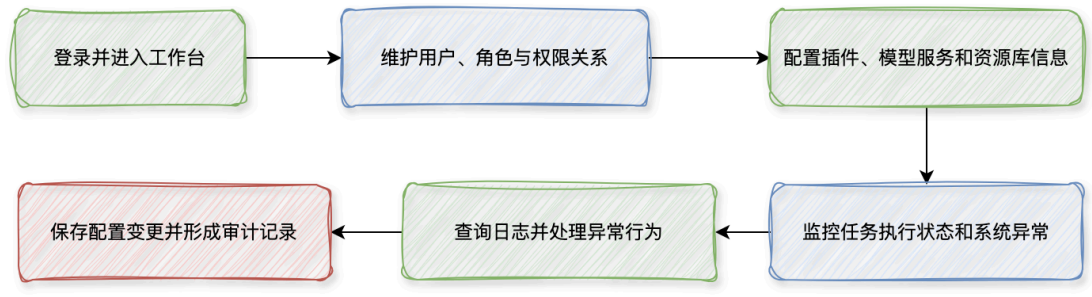


图 4: 系统管理员业务流程图

系统管理员的业务流程主要围绕系统治理展开。管理员登录后进入后台管理工作台，对用户、角色和权限关系进行维护，并对 Blender 插件、模型服务接口以及资源库状态进行配置和检查。在系统运行过程中，管理员需要监控任务执行情况、识别异常任务和潜在故障，并通过日志查询定位问题来源；对于权限越界、模型接口异常、资源失效等问题，管理员应及时进行处理；最终，系统对后台配置变更和关键操作形成审计记录。该流程体现的是系统持续运行和维护保障能力。

3.2.4 场景生成与资产替换流程

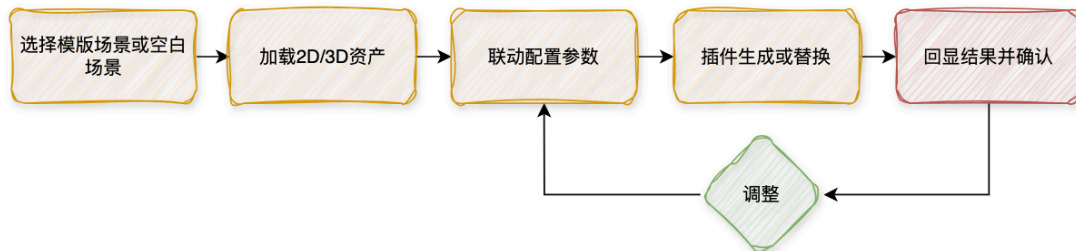


图 5: 场景生成与资产替换流程图

该流程是场景建模师业务流程中的第一类核心细化流程，主要面向基础城市场景的快速创建与对象替换。用户从模板或空白场景开始，调用资产库中的道路、建筑、树木和庭院等资源，并根据当前模型类别联动配置相关参数。例如，当用户选择某类道路模型时，系统应同步展示与之适配的宽度、材质和配套绿化参数；当用户切换某类庭院风格时，相关装饰资源和密度配置也应同步更新。参数确认后，主系统将任务提交给 Blender 插件执行，插件返回结果后，系统在视图中回显变化，使用户能够立即判断当前替换是否符合预期。该流程强调的不是“一次性生成完成”，而是支持多轮替换、比较和细化。

3.2.5 布局与生态控制流程

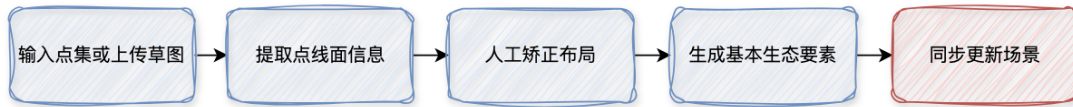


图 6: 布局与生态控制流程图

该流程是场景建模师业务流程中的第二类核心细化流程，重点解决城市结构调整和外围生态塑造问题。与资产替换相比，这一流程更关注空间关系本身。系统允许用户手动输入精确点线集，以满足较高精度的布局控制需求；同时，也支持通过草图快速描述总体道路走向、区域边界和功能分区。系统对草图进行点线面提取后，应生成一个可供确认的中间布局结果，而不是直接覆盖原场景。待用户确认或修正布局后，系统再据此生成山体、盆地、湖泊和动态船只等生态对象。该流程的关键价值在于，用户不仅能够“放置对象”，还能够“塑造空间结构和环境氛围”。

3.2.6 LLM 智能交互与仿真分析流程

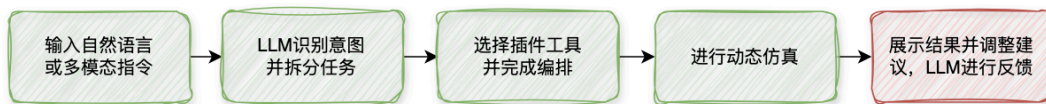


图 7: 智能交互与仿真分析流程图

该流程是场景建模师业务流程中的第三类核心细化流程，同时也是题目要求中的创新功能所在。用户可以直接输入类似“将城市外围改成环湖布局，并增加若干船只和沿岸绿地”的自然语言指令，或者结合草图和参数表单形成多模态输入。系统通过大语言模型识别其中涉及的对象、属性、数量、空间关系和修改意图，并将复杂描述拆解为多个结构化子任务，再根据任务类型自动选择对应插件能力执行。任务完成后，系统通过车辆、人群和船只等动态对象展示场景效果，帮助场景建模师和行业分析师观察不同方案的动态差异。该流程体现了系统由传统参数驱动模式向意图驱动模式的延伸，也是本报告中最能体现系统创新性的业务环节。

3.2.7 LLM 驱动的任务转换过程说明

为了使智能交互需求表达得更具体，系统还需要明确从原始输入到可执行任务之间的转换过程。对于自然语言或多模态输入，主系统不应直接把原始文本交给插件执行，而应经过“输入接收、上下文补充、意图识别、参数抽取、冲突检查、结构化任务生成、用户确认、插件调用”这样一条相对完整的处理链路。该链路的存在，决定了系统能够在保持交互自然性的同时，仍然维持需求上的可解释性和可控性。

在这一过程中，大语言模型首先要结合当前场景上下文理解用户表达的修改目标。例如，当用户输入“在东侧增加沿湖道路”时，系统不仅要识别“道路”这一对象类型，还要理解“东侧”对应的空间区域、“沿湖”对应的空间关系，以及该要求是否依赖已有湖泊对象。随后，主系统还应依据资产库状态、当前场景边界和插件能力范围，对模型给出的任务建议进行约束检查。若用户输入与现有场景条件不一致，例如当前场景不存在可供“沿湖”约束依附的水体区域，系统不应直接执行，而应提示用户补充条件、先生成湖泊，或改写任务。

需求分析中单独说明这一路径的原因，在于它直接影响后续接口设计和错误处理设计。如果没有结构化任务生成与确认环节，模型输出就只能作为模糊建议存在，难以真正进入系统执行链路；如果没有冲突检查和异常提示，模型理解的不确定性就会直接放大为场景结果的不确定性。因此，LLM 在本系统中并不是孤立的“智能黑盒”，而是必须与场景上下文、资源约束、插件能力和用户确认机制共同组成一个闭环任务转换过程。

四、 数据需求

(一) 数据需求描述

数据需求部分不仅要说明系统包含哪些数据，还要说明这些数据在系统中承担什么作用、从哪里产生、被谁使用以及如何保持一致。对于智能城市生成系统而言，数据既包含传统管理系统中的结构化数据，也包含三维场景系统特有的半结构化和非结构化数据。

本系统的数据可划分为以下几类：

1. 用户与权限数据：包括用户编号、角色信息、权限集合、状态和登录记录，用于控制系统访问边界。
2. 项目与场景数据：包括项目元信息、场景版本、场景状态、场景文件位置和生成历史，用于支撑项目化管理。
3. 资产资源数据：包括道路、建筑、树木、庭院、水体、山体、动态对象等资源及其分类、标签和版本。
4. 布局与输入数据：包括点集、线集、草图图像、参数表单和中间布局结果，用于表达用户设计意图。
5. 智能任务数据：包括原始文本指令、多模态输入摘要、模型解析结果、任务拆解结构、执行状态和错误信息。
6. 仿真与结果数据：包括车辆、人群、船只等对象的行为参数、仿真结果快照和导出结果。
7. 日志与审计数据：包括操作日志、任务日志、错误日志和配置变更日志，用于系统维护和问题追踪。

从系统运行需求来看，数据并不是孤立存在的存储对象，而是直接服务于任务执行、结果展示、权限控制和系统维护的基础支撑。因此，本节在组织数据需求时，不仅关注数据本身的类型划分，也关注数据与功能、接口、质量约束之间的对应关系。这样处理的好处在于，后续在进行接口设计、数据库设计和测试设计时，可以更清楚地判断每类数据分别服务于哪些系统能力，以及它们在系统运行中的作用边界。

1. 用户与权限数据

用户与权限数据是系统最基础的一类数据，用于界定“谁可以访问系统、谁可以执行哪些操作、谁可以查看哪些结果”。该类数据通常由账号信息、角色信息、权限集合、登录状态和安全日志构成。对这类数据的要求主要有三点：第一，必须具备唯一性和一致性，例如用户编号不能重复，角色与权限之间的映射不能冲突；第二，必须具备可审计性，尤其是权限修改、账号停用和后台配置变更等操作；第三，必须与系统展示层保持一致，即用户在界面上看到的可用菜单应与其权限完全一致，避免出现“界面可见但不可操作”或“功能不可见但实际上已授权”的矛盾情况。

2. 项目、场景与版本数据

项目与场景数据构成系统的核心业务数据。项目数据用于组织多个场景和任务过程，场景数据则描述某一时刻的城市空间状态、对象索引和资源引用关系。由于智能城市生成系统具有明显的迭代特征，同一场景通常会经历多轮修改和局部替换，因此版本管理要求非常重要。系统应能够区分不同版本的场景状态，记录每次生成或修改对应的时间、操作者和任务来源。这样做的意义在于，一旦后续需要回滚、对比或解释结果差异，系统能够提供足够依据，而不是只有一个被覆盖后的最终结果。

3. 资产与资源数据

资产与资源数据是系统完成三维生成的物质基础，包括道路、建筑、树木、庭院、水体装饰、山体资源以及车辆、人群和船只等动态对象资源。此类数据除了需要记录路径和名称外，还应至少包含资源类别、资源标签、适用场景、资源版本和状态信息。对于需求分析而言，资源数据的重点不是简单“能存资源”，而是系统需要具备足够的资源检索和匹配能力，使得用户在进行风格选择或参数联动时，能够调用与当前场景相容的资源，而不会出现明显不匹配的对象组合。

4. 布局、草图与中间结果数据

与一般管理系统相比，本系统特有的数据类型在于布局和草图输入。布局点线集、草图图像以及由草图识别得到的中间结果，属于表达设计意图的重要数据。它们往往不直接等同于最终场景，却在任务生成过程中起着决定性作用。系统对这类数据的要求主要体现在两个方面：其一，需要保留足够的中间结构，以便用户在识别错误时进行人工修正；其二，需要能够将这些输入与项目、任务和最终场景版本建立关联，确保用户能够理解“这个场景结果是由哪份草图、哪组点线数据或哪套参数驱动产生的”。

5. 智能任务与日志审计数据

智能任务数据和日志审计数据共同支撑系统的可解释性与可维护性。前者记录用户原始自然语言输入、多模态输入摘要、模型解析结果、任务拆解结构和执行状态；后者记录关键操作、配置变更、插件执行结果和错误信息。对于引入 LLM 的系统而言，若缺少智能任务记录，就很难解释模型为何生成某项任务；而若缺少日志审计数据，则很难定位插件异常、资源失效或权限问题。因而，这两类数据虽然不直接决定场景外观，却是系统稳定运行不可缺少的组成部分。

进一步来看，智能任务数据本身还应具有较强的过程性，而不能只保留一个最终执行结果。较为完整的任务记录至少应包括原始输入内容、输入来源类型、模型解析时间、参与解析的上下文摘要、抽取出的对象与属性、推断出的空间关系、生成的结构化任务列表、用户确认结果以及最终执行状态。只有保留这些中间信息，系统才能在后续出现“结果与预期不符”时，追溯问题究竟发生在用户表达不清、模型理解偏差、约束检查缺失，还是插件执行失败这一环节。

对于多模态输入，数据关联关系尤其重要。若一项任务同时包含自然语言、草图和参数表单，则系统应能够说明哪些字段来自文本解析，哪些约束来自草图提取，哪些参数来自用户明确配置。这样做的意义在于提高任务解释结果的透明度，避免用户在看到最终场景后无法判断“系统到底依据了哪部分输入”。

(二) 数据流程图

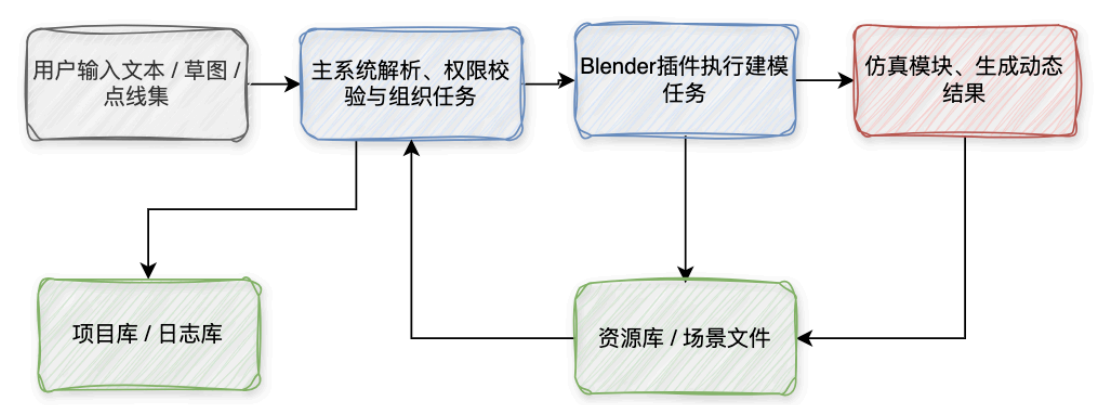


图 8: 系统数据流程图

如图所示，用户输入首先流入主系统，主系统执行权限判断、输入规范化、参数补全和任务封装，然后将任务发送给 Blender 插件执行。插件在执行时需要访问资产库与场景文件，并将生成结果返回主系统；若任务涉及动态对象，则仿真模块继续在生成结果基础上输出动态表现。与此同时，项目库和日志库保存项目状态、任务记录和错误信息，从而实现任务可追踪、结果可回看和错误可定位。

(三) 数据字典

1. 核心数据项

表 3: 核心数据字典

数据项	含义	类型	来源	说明与约束
User	用户信息	结构化	登录与管理模块	记录编号、姓名、角色、状态；编号唯一。
Role	角色信息	结构化	权限管理模块	至少包含建模师、分析师、管理员三类角色。
Permission	权限集合	结构化	权限配置模块	与角色关联，不允许存在未定义权限。
Project	项目信息	结构化	项目管理模块	记录项目名、创建者、创建时间和状态。
Scene	场景对象	半结构化	场景生成模块	记录场景版本、结构对象索引和文件关联信息。
Asset	资产资源	半结构化	资产库	包含类别、标签、路径、版本和适用范围。

续下页

数据项	含义	类型	来源	说明与约束
SketchImage	草图图像	非结构化	用户上传	用于布局提取，应与项目或任务关联保存。
LayoutSet	布局点线集	半结构化	手动输入或图像提取	描述道路、边界、区域和控制点。
TaskConfig	任务参数	半结构化	表单输入或智能模块	记录具体生成参数及其来源。
LLMTask	智能任务	半结构化	智能交互模块	保存原始指令、解析结果、状态和异常信息。
SimObject	仿真对象	半结构化	仿真模块	记录车辆、人群、船只的行为参数。
OperationLog	操作日志	结构化	日志模块	保存操作者、动作、时间、结果和目标对象。

2. 数据需求补充说明

对于用户与权限数据，系统应强调准确性和一致性，确保不同角色仅访问被授权内容。对于项目与场景数据，系统应强调版本可回溯性，因为场景生成和编辑通常是多轮迭代过程。对于资产数据，系统应强调分类清晰和版本清楚，以避免资源混用。对于智能任务数据，系统应强调可解释性，即至少能让用户知道原始指令如何被解析成结构化任务。对于日志数据，系统应强调完整性和可检索性，以支持后续排错和审计。

表 4: 主要数据访问关系表

数据类别	主要访问角色	访问目的
用户与权限数据	系统管理员	完成账号维护、权限配置与安全审计。
项目与场景数据	建模师、分析师、管理员	支撑场景生成、结果查看、版本管理与追溯。
资产与资源数据	建模师、管理员	选择资源、维护资源库、控制资源版本。
布局与草图数据	建模师	输入设计意图、修正布局结构、复用中间结果。
智能任务数据	建模师、分析师、管理员	查看解析结果、解释任务来源、定位模型异常。
仿真与结果数据	建模师、分析师	查看动态效果、比较方案、导出展示结果。
日志与审计数据	系统管理员	追踪问题来源、审查关键操作、支持维护工作。

五、 功能需求

(一) 功能划分

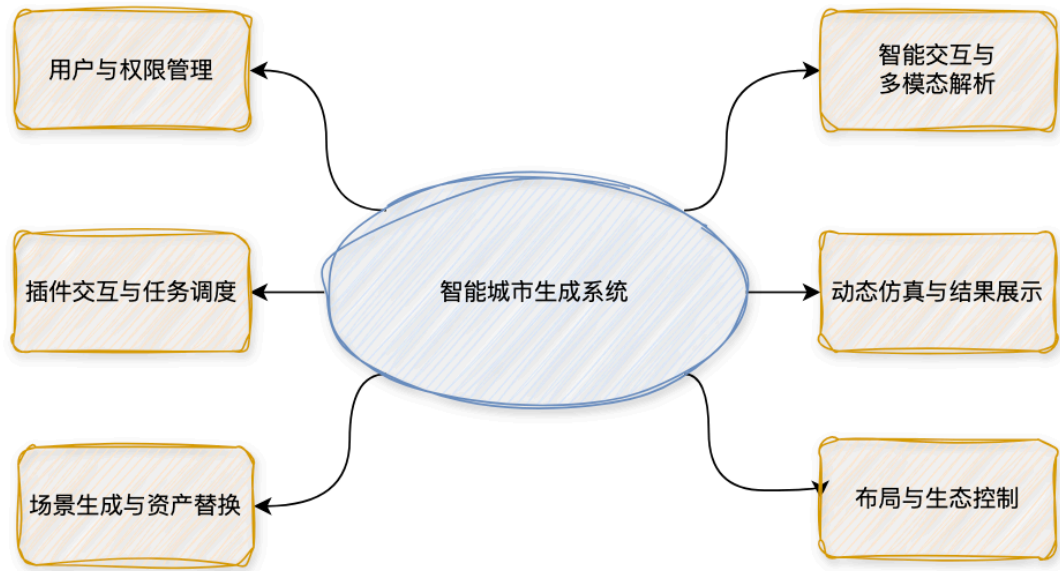


图 9: 系统功能结构图

本文采用“按子系统/功能模块划分”的组织方式。这样划分的根本原因在于，本系统的主要能力边界较为清晰，不同功能之间在职责上也具有较强区分度。例如，用户与权限管理侧重系统治理，插件交互与任务调度侧重执行链路，场景生成与布局控制侧重建模能力，动态仿真与结果展示侧重结果消费。按照功能模块展开描述，能够更清楚地反映系统内部各部分的职责分工，也更有利于后续进行模块化设计、接口划分和测试项分解，避免把不同性质的需求混杂在同一层次中。

(二) 功能描述

1. 用户与权限管理

在系统整体结构中，用户与权限管理模块承担的是“系统入口控制与行为约束”的基础角色。与具体业务功能相比，该模块并不直接参与场景生成，但它决定了不同用户在系统中的可见范围与可操作边界，是系统安全性与规范性的核心保障。

从使用场景来看，本系统存在明显的角色分工，例如建模师关注场景构建，分析师关注结果展示，而管理员关注系统维护。因此，权限控制不应仅停留在登录层面，而需要贯穿到菜单展示、数据访问和功能执行等多个环节。换言之，系统不仅需要判断“用户是谁”，还需要明确“用户能看到什么、能做什么”。

在设计上，该模块需要重点处理两个问题：一是权限粒度的划分，应在角色级与功能级之间取得平衡，既避免过于粗粒度导致权限混乱，也避免过细导致管理复杂；二是操作可追踪性，所有涉及权限变更和关键操作的行为都应被记录，以支持后续审计与问题定位。

因此，该模块不仅是一个认证机制，更是整个系统运行秩序的基础约束层。

编号	功能名称	参与角色	功能描述
FR-01	用户登录认证	全部角色	系统应支持账号登录,并在认证通过后建立会话状态;认证失败需明确提示。
FR-02	角色识别与菜单装载	全部角色	系统应根据用户角色自动装载菜单与数据访问范围,防止越权操作。
FR-03	权限配置与调整	管理员	支持维护角色与权限关系,并记录权限变更日志。
FR-04	日志审计查询	管理员	记录登录、权限修改、任务提交等操作,并支持日志检索。

2. 插件交互与任务调度

插件交互与任务调度模块构成了系统“逻辑层”与“执行层”之间的关键连接。在本系统架构中,主系统负责组织任务与用户交互,而具体的三维场景生成则依赖 Blender 插件完成。因此,该模块的稳定性直接决定了系统功能能否真正落地。

从数据流角度看,该模块的核心作用是将多来源输入(如文本、草图和参数)转化为统一的任务描述,并在主系统与插件之间建立可靠的传递机制。这一过程不仅包括任务下发,还包括状态回传、错误反馈以及执行结果同步。

在设计时,需要特别关注任务执行的异步特性。由于场景生成和资源替换通常具有一定耗时,系统不应采用阻塞式调用,而应支持任务排队、进度反馈和中间状态展示。同时,当插件执行失败或连接异常时,系统应能够及时捕获并反馈错误,而不是简单返回空结果。

此外,该模块还需要具备一定的解耦能力,即主系统不应依赖某一具体插件实现。通过统一的任务接口设计,可以使插件能力在后续扩展或替换时保持相对独立,从而提高系统整体的可扩展性。

编号	功能名称	参与角色	功能描述
FR-05	插件连接管理	建模师、管理员	系统应检测插件连接状态、版本信息和可用能力,并在异常时进行提示。
FR-06	任务封装与下发	建模师、分析师	系统应将文本、草图、点线集和参数配置统一封装为可执行任务并发送给插件。
FR-07	执行状态回传	建模师、分析师、管理员	插件执行过程中应回传任务进度、成功状态、失败原因和必要日志信息。

表 7: 插件交互与任务调度模块规格说明

项目	说明
输入	结构化任务、资源路径、参数配置、插件状态查询请求
处理	插件状态检测、任务封装、任务下发、进度回传、异常收集
输出	执行状态、执行结果、失败原因、任务日志
异常与约束	插件离线时不得提交正式任务；任务失败时需返回可理解原因；关键状态需回写日志库

3. 场景生成与资产替换

场景生成与资产替换模块是系统最核心的功能承载部分，其直接决定了用户能否高效构建和调整城市空间。与布局模块不同，该模块更侧重“对象层”的操作，即如何选择合适的资源并将其合理地组织到场景中。

在实际使用过程中，用户往往并非从零开始建模，而是基于已有结构不断进行替换与优化。因此，该模块需要支持渐进式生成过程，而不是一次性完成所有内容。具体而言，系统应允许用户在已有场景基础上多次调整资源类型、风格参数和局部区域，而不破坏整体结构。

从设计角度看，该模块的难点在于“资源匹配与参数联动”。不同资产之间往往存在隐含关系，例如某类道路适配特定宽度或绿化风格，如果系统不能正确处理这些关系，就容易出现风格不一致或结构不合理的问题。因此，资源选择不应是孤立操作，而应伴随参数联动与约束检查。

此外，该模块还应强调结果的可编辑性。生成结果不应被视为终态，而应始终处于可调整状态，从而支持用户在多轮操作中逐步逼近理想方案。

编号	功能名称	参与角色	功能描述
FR-08	基础场景创建	建模师	系统应支持从模板场景或空白场景出发生成基础城市结构。
FR-09	资产加载与浏览	建模师	系统应支持道路、建筑、树木、庭院等 2D/3D 资产的分类加载和属性浏览。
FR-10	资产替换与重生 成	建模师	系统应支持对现有场景中的对象执行局部或批量替换，并尽量保持原布局结构稳定。
FR-11	联动参数配置	建模师	用户选择某类资源后，系统应联动展示与其匹配的道路、树木、庭院等配置项。

表 9: 场景生成与资产替换模块规格说明

项目	说明
输入	模板场景、空白场景、资产选择结果、风格参数、替换范围
处理	资源检索、参数联动、对象实例化、局部替换、结果回显
输出	新场景结果、局部替换结果、更新后的场景版本信息
异常与约束	替换操作应尽量保持原布局稳定；资源缺失时需阻止执行并提示；生成结果应可继续编辑

4. 布局与生态控制

与资产替换模块主要关注对象选择不同，布局与生态控制模块更强调空间结构与环境语义的构建。在城市建模过程中，单纯堆叠对象往往难以形成合理场景，而需要通过布局关系来组织整体空间。

该模块的一个重要特点在于输入形式的多样性。一方面，专业用户可以通过点线集实现精确控制，从而满足较高精度的设计需求；另一方面，普通用户则更倾向于通过草图表达整体结构。因此，系统需要同时支持结构化输入与半结构化输入，并在两者之间建立转换机制。

在生态生成方面，本模块不仅负责添加环境元素，还需要保证这些元素与现有布局保持一致。例如，湖泊的位置应与道路结构协调，船只的生成应依赖水域存在。这意味着生态生成并非简单的“对象添加”，而是一种基于空间约束的生成过程。

从交互角度看，该模块还承担“中间结果展示”的职责。对于草图解析等自动过程，系统应优先输出可供确认的布局结果，而不是直接覆盖场景。这种设计能够显著降低错误传播风险，也是提升系统可控性的关键手段。

编号	功能名称	参与角色	功能描述
FR-12	点集精确布局编辑	建模师	支持手动输入点集、线集及控制参数，实现道路或区域的精确布局调整。
FR-13	草图信息提取	建模师	支持从草图中自动提取点线面信息，并允许用户对识别结果进行修正。
FR-14	生态要素生成	建模师	支持在指定区域生成山体、盆地、湖泊等生态对象。
FR-15	动态船只配置	建模师、分析师	支持在水域区域配置船只，并设置路径、数量及速度参数。

表 11: 布局与生态控制模块规格说明

项目	说明
输入	点集、线集、草图图像及生态对象配置参数。
处理	点线面提取、人工修正、布局更新、生态对象生成及船只配置。
输出	更新后的布局结构、生态对象结果及场景版本信息。
异常与约束	草图识别结果必须允许修正；生态生成需与场景边界协调；水域缺失时禁止生成船只。

5. 智能交互与多模态解析

智能交互与多模态解析模块是本系统最具创新性的部分，其核心目标在于降低用户表达成本，使复杂建模任务能够通过自然方式完成。相比传统参数驱动方式，该模块引入了意图驱动机制，即系统首先理解“用户想做什么”，再将其转化为可执行任务。

在实际应用中，用户输入往往具有不完整性和歧义性。例如，“增加一条沿湖道路”这一指令隐含了区域、对象和空间关系等多个信息维度。因此，该模块需要通过上下文补充与语义推断，将原始输入扩展为结构化描述。

设计上的关键在于控制不确定性。模型解析结果不应直接执行，而应通过任务解释形式展示给用户，使其能够确认或修正。这一过程不仅提升了系统透明度，也避免了错误理解直接影响最终结果。

此外，多模态输入带来的一个挑战在于信息融合。例如，当文本与草图同时存在时，系统需要判断各自的权重与约束来源。为此，系统应保留输入来源信息，使用户能够清晰了解每一部分结果的依据。

总体而言，该模块不仅是一个“智能入口”，更是连接用户意图与系统执行链路的核心桥梁。

编号	功能名称	参与角色	功能描述
FR-16	自然语言控制	建模师、分析 师	支持通过自然语言描述场景创建或修改需求，并输出可解释的任务结果。
FR-17	多模态指令解析	建模师	支持融合文本、草图和参数表单，解析为统一任务。
FR-18	结构化任务生成	建模师	将解析结果转换为结构化任务，包括对象、数量、空间关系与参数建议。
FR-19	歧义提示与二次 确认	建模师、分析 师	输入存在歧义或冲突时，提示用户补充信息或进行确认。
FR-20	自动插件调用	建模师	根据确认任务自动调用插件并执行。
FR-21	解析过程留痕	建模师、管理 员	记录从输入到执行的完整过程，支持追踪与审计。

表 13: 智能交互与多模态解析模块规格说明

项目	说明
输入	自然语言、草图、参数表单及场景上下文。
处理	意图识别、对象抽取、关系解析、任务拆解、冲突检测、用户确认与插件编排。
输出	结构化任务、解释结果、确认记录及执行结果。
异常与约束	存在歧义需提示确认；模型失败需回退；未确认任务不得执行。

6. 动态仿真与结果展示

动态仿真与结果展示模块的主要作用在于将静态场景转化为可观察、可比较的动态结果，从而提升系统在分析与展示场景中的应用价值。与建模模块不同，该模块更关注“结果表达”而非“结果生成”。

在实际应用中，静态场景往往难以反映空间设计的实际效果，例如道路布局是否合理、区域是否具备通行能力等。因此，通过车辆、人群和船只等动态对象，可以直观展示场景的行为特征。

该模块在设计上需要兼顾表现力与计算开销。由于系统定位于展示与分析，而非高精度仿真，因此应采用规则驱动或简化模型，以保证运行效率。同时，仿真过程应支持参数调节，使用户能够快速对比不同方案下的表现差异。

此外，结果展示不仅限于实时预览，还应支持导出与对比功能。例如，通过截图或结果文件，用户可以将不同方案用于报告或展示。这一能力使系统从“建模工具”扩展为“分析与表达工具”。

因此，该模块在整体系统中承担的是“结果放大器”的角色，使已有场景具备更强的解释能力和展示价值。

编号	功能名称	参与角色	功能描述
FR-22	车辆动态模拟	建模师、分析 师	在道路网络上配置车辆并展示运动行为。
FR-23	人群动态模拟	建模师、分析 师	在公共区域配置行人并展示行为轨迹。
FR-24	仿真结果预览与 导出	建模师、分析 师	支持仿真结果预览、截图与导出。

表 15: 动态仿真与结果展示模块规格说明

项目	说明
输入	场景结构、道路、水域、仿真参数。
处理	配置车辆、人群、船只并执行仿真，生成展示结果。
输出	动态演示结果、对比结果及导出文件。
异常与约束	仿真仅用于展示分析；缺少必要结构时拒绝执行。

(三) 功能补充说明

为使功能需求更具可测试性，现补充说明如下：

- 1. 对于 FR-11，系统不应仅显示固定参数列表，而应随资源类别变化动态刷新参数候选项。
- 2. 对于 FR-13，自动提取结果必须允许人工修订，否则识别误差会直接导致布局错误。
- 3. 对于 FR-16 至 FR-18，系统应保存从原始指令到结构化任务的转换结果，以使用户确认和后续追踪。
- 4. 对于 FR-19 至 FR-21，系统重点在于可视化演示和方案比较，而非工程级仿真精度。

(四) 关键功能场景说明

1. 典型场景一：基于模板快速生成城市街区

场景建模师选择某一模板场景后，从资产库中选择道路和建筑资源，并通过联动参数配置树木和庭院风格。系统将相关选择封装为结构化任务，提交给 Blender 插件执行，随后返回一个包含道路、建筑和绿化对象的初始街区场景。该场景应支持继续替换局部资源、调整布局和导出结果。该场景说明了系统最基础也最常用的一条功能链路。

2. 典型场景二：基于草图调整城市外围布局

用户上传一张表达外围道路、水域和绿地关系的草图，系统提取草图中的点线面信息，并生成中间布局结果供用户检查。用户确认后或修正后，系统更新布局结构并在指定区域加入湖泊、山体 and 船只等生态对象。该场景体现了系统处理半结构化输入和复杂空间调整的能力。

3. 典型场景三：通过自然语言触发插件自动建模

用户输入“在城市东侧新增滨水公园，并增加沿湖道路和两艘动态船只”等自然语言指令。系统利用 LLM 提取出区域、对象、数量与关系信息，并生成若干结构化子任务，随后自动调用相应插件能力执行。执行完成后，系统返回修改后的场景结果及动态展示结果。该场景体现了系统创新功能的完整使用方式。

在该典型场景下，系统还应向用户展示必要的任务解释信息。例如，系统可将原始指令解释为“在东侧区域生成滨水公园对象”“在靠近水域边界处生成沿湖道路”“在指定水域中加入两艘船只”等若干子任务，并标明相关对象的数量与空间约束。若当前场景中不存在可供停放船只的水域区域，系统应在执行前给出提示，而不是直接忽略该子任务。这样处理的原因在于，LLM 的价值不只体现在“自动理解”，更体现在“让用户理解系统是如何理解自己的”。

对于需求分析而言，这一场景还说明了模型能力与系统约束必须同时发挥作用。自然语言输入带来了更低的表达门槛，但也带来了更高的不确定性；而结构化任务生成、场景上下文检查和

用户确认机制，正是系统控制这种不确定性的关键环节。只有在这一闭环存在的前提下，LLM 功能才能被视为真实可用的软件能力，而不是停留在概念演示层面。

六、性能/非功能需求

（一）系统的准确性

系统应尽量保证生成结果与用户确认后的输入意图保持一致。对于参数表单输入，系统应确保插件执行所用参数与界面确认参数一致；对于自然语言输入，系统应在存在歧义时给出任务解释或二次确认，而不应直接生成不可预期结果；对于草图识别结果，系统应支持用户在执行前进行校正。准确性的核心不是“每次都自动完美生成”，而是“系统行为应尽可能与用户确认后的输入相一致”。

为了使准确性需求具备更强的可检查性，系统在实现时至少应满足以下要求：其一，参数型任务应保证用户确认参数与插件执行参数之间不存在无提示偏差；其二，草图识别结果在正式执行前必须展示中间结果供用户确认；其三，LLM 解析结果应输出结构化解释，使用户能够判断系统是否正确理解了输入意图。若系统无法满足这些基本条件，则即使最终场景表面上“看起来合理”，也不能认为其准确性需求已经得到满足。

（二）模型解析的可解释性与容错性

由于本系统引入了 LLM，因此还需要单独强调模型解析过程的可解释性与容错性。可解释性的核心要求在于，系统不应只返回一个“已经生成完成”的结果，而应让用户能够看到模型将原始输入理解成了哪些对象、关系和任务，并允许用户在必要时对其进行确认、修正或拒绝。尤其是在输入较长、语义较复杂或包含多模态信息时，若系统缺少解释层，用户就很难判断最终结果偏差究竟来自自身表达、模型理解还是执行阶段。

容错性的核心要求在于，系统必须承认模型解析存在不确定性，并在需求层面给出处理方法。对于语义歧义、上下文不足、场景条件不满足、模型超时或解析失败等情况，系统应提供明确提示、保留原始输入、允许重新解析，并尽量避免对已有场景结果造成破坏。若模型服务暂时不可用，系统仍应允许用户回退到参数表单、点线集或草图驱动的传统交互方式，而不是使整个系统主要功能完全失效。

从质量要求的角度看，可解释性和容错性实际上决定了 LLM 功能能否真正被工程化使用。一个只在理想输入下表现良好的模型，并不能直接等同于一个满足软件需求的模块；只有当系统同时具备解释输出、异常提示、状态回传和回退路径时，模型能力才符合课程要求中“可测试、可维护、可验证”的需求表达标准。

（三）系统的及时性

系统应具备可接受的响应速度。轻量级操作如页面切换、资产浏览、参数调整和结果刷新应较快完成；较重任务如大场景生成、批量替换和仿真执行则应提供进度反馈和异步回传能力。及时性的重点不仅在于快，还在于让用户清楚系统正在做什么、是否完成、失败原因是什么。

结合本系统的应用场景，及时性可进一步细化为两类要求。第一类是交互及时性，即普通页面切换、资源检索、参数刷新和结果预览等操作应尽量保持流畅，不应频繁出现长时间无响应状态。第二类是任务及时性，即对于需要调用 Blender 插件或模型服务的复杂任务，系统应提供明确的任务状态展示，例如“排队中”“执行中”“执行成功”“执行失败”等。这样即使任务本身耗时较长，用户仍然能够对系统状态保持清晰认知。

(四) 系统的可扩充性

系统应支持在后续版本中新增资产类别、生态对象、插件能力和模型服务接口。模块之间应保持相对清晰的边界，资产配置应尽量以可扩展方式管理，模型服务应支持替换和新增。这样一来，后续若要增加新的场景风格、生态元素或更强的语言模型，不必从头重构系统。

可扩充性之所以重要，是因为本系统天然具有持续演化特征。随着场景类型丰富、资源库扩展和模型能力迭代，系统不可能长期停留在初始版本。如果需求分析阶段不提前考虑扩展问题，那么后续每新增一种资源类型、每更换一次模型接口、每增加一种仿真对象，都可能引发大范围改动。因此，在需求层面强调模块边界清晰和接口可替换性，有助于降低未来功能扩展的成本。

(五) 系统的易用性

系统应对不同角色提供清晰的操作路径。场景建模师侧重建模效率与参数可控性，因此界面应突出任务流和局部修改能力；行业分析师侧重结果观察和导出，因此界面应减少复杂参数暴露；系统管理员侧重后台治理，因此界面应突出权限、日志和系统状态。对于高风险操作、执行失败和异常输入，系统必须给出明确提示。

易用性并不意味着界面元素越少越好，而是意味着系统界面结构应与用户任务路径保持一致。对于建模师，常用生成操作、参数联动与结果回看应尽量处于同一工作流中，避免在多个页面之间来回切换。对于分析师，系统应尽可能直接展示关键结果，而不是要求其先学习大量建模术语。对于管理员，后台配置项和日志检索条件应保持命名清楚、状态明确，降低误配置风险。

(六) 系统的易维护性

系统应具备良好的可维护特征，包括明确的模块边界、统一的配置入口、必要的日志记录和可追踪的错误信息。系统管理员应能够查看任务执行日志、模型服务状态、插件异常记录 and 用户关键操作。若未来发生插件升级、模型替换或资源库调整，系统应尽量通过配置和适配完成修改，而不是大范围重写。

从实际运行角度看，维护难度往往不体现在“功能能不能运行”，而体现在“出问题后能不能快速定位原因”。因此，本系统在维护性上的需求重点应放在问题可诊断性上，包括任务编号可追踪、关键操作有日志、异常返回有上下文、资源和版本信息可查询等。只有这样，当系统出现模型接口失效、插件执行异常或资源缺失时，维护人员才能在较短时间内完成定位和处理。

(七) 系统的标准性

系统应尽量采用通用数据格式与稳定接口约定。例如，任务配置和解析结果应采用结构清楚的字段组织方式，导出文件应支持常见格式，系统与外部模块之间的交互应遵循明确的输入输出规则。标准性的意义在于降低后续集成与维护成本。

(八) 系统的先进性

系统应体现一定的技术先进性，主要体现在对程序化建模、动态仿真、多模态输入和大语言模型解析的综合运用上。需要强调的是，先进性不等于堆砌热门技术，相关技术引入应服务于提高场景生成效率、改善用户交互和降低建模门槛。

(九) 系统的安全性

系统应保证不同角色之间的功能和数据隔离。用户认证、权限分配、模型服务配置、日志访问和后台治理操作都应受到控制。关键操作应有日志记录，必要时应提供确认步骤。系统还应避

免未授权用户直接访问场景文件、任务配置和后台接口。

对于本系统而言，安全性的重点不仅是传统意义上的账号安全，还包括任务执行安全和资源访问安全。例如，普通分析师不应具备修改模型服务配置的能力，普通建模师也不应直接操作高权限后台模块；此外，资源库和场景文件虽然主要面向系统内部使用，但仍应避免通过非授权方式被任意替换或删除。只有将用户、任务、资源和配置四个层面的安全边界同时考虑进去，安全性要求才是完整的。

(十) 系统的可靠性与可移植性

系统应在插件执行失败、模型超时、资源缺失或输入不合法时给出明确提示，并尽量保护已保存的项目状态不被破坏。系统还应具备一定可移植性，能够在支持 Blender 的主流桌面操作系统环境中部署和运行，并兼容常见资源格式和输入方式。

可靠性强调系统在异常情况下的可预期行为，可移植性强调系统在不同目标环境中的适应能力。对于本项目而言，两者都非常重要：若系统一旦出现任务失败就导致场景损坏，则即使功能丰富也难以真正使用；若系统仅能在单一环境中运行，则后续展示和部署范围会受到明显限制。因此，本系统应至少保证异常可提示、已保存结果不被随意覆盖、关键环境依赖相对明确，并尽量避免对单一平台特性形成过度绑定。

表 16: 非功能需求汇总表

编号	类别	需求说明
NFR-01	准确性	生成结果应与用户确认后的文本、草图和参数配置尽量保持一致。
NFR-02	及时性	轻量操作应快速反馈，长任务应提供进度回传和失败提示。
NFR-03	可扩充性	应支持新增资产、生态对象、模型接口和插件能力。
NFR-04	易用性	应为不同角色提供清晰操作路径，并对关键操作给出提示。
NFR-05	易维护性	应具备清晰模块边界、统一配置入口和可追踪日志。
NFR-06	标准性	应尽量采用通用数据格式与明确接口约定。
NFR-07	先进性	应体现多模态输入、LLM 解析和程序化建模协同能力。
NFR-08	安全性	应实现角色隔离、关键操作记录和后台访问控制。
NFR-09	可靠性	执行失败或异常时应给出明确反馈并尽量保护项目状态。
NFR-10	可移植性	应支持主流桌面环境与常见资源格式。
NFR-11	可解释性	LLM 解析结果应向用户提供结构化解释，并支持确认或修正。
NFR-12	容错性	模型超时、歧义解析或上下文冲突时应支持提示、回退与重新解析。

七、 系统运行要求

(一) 硬件配置要求

由于系统涉及三维场景生成、资源加载和动态仿真展示，其硬件要求高于普通管理型软件。尤其在 Blender 插件执行复杂任务时，CPU、内存和显卡性能都会直接影响生成效率 and 操作流畅度。

表 17: 硬件配置要求

项目	最低要求	建议配置
处理器	四核 CPU	八核及以上 CPU
内存	8GB	16GB 及以上
显卡	支持 OpenGL 的独立显卡	图形处理能力更强的独立显卡
存储空间	10GB 可用空间	30GB 及以上可用空间
网络环境	稳定互联网连接	低延迟宽带环境

上述硬件要求可理解为系统顺利运行的基本条件。其中，CPU 和内存主要影响任务组织、模型调用和插件执行时的整体吞吐能力；显卡则直接影响 Blender 视图渲染和场景预览效果；存储空间不仅关系到软件本身安装，也关系到资产资源、项目文件和历史版本的保留能力。若运行环境低于最低要求，系统仍可能启动，但在复杂场景生成、连续替换和动态仿真环节中容易出现显卡顿和等待时间过长的问題。

(二) 软件配置要求

从软件运行环境角度看，系统不仅需要 Blender 及其插件能力，还需要主系统运行环境、数据存储环境和模型服务环境协同支持。

表 18: 软件配置要求

项目	要求说明
操作系统	支持 Windows、Linux 或 macOS 中能够正常运行 Blender 的环境
Blender 环境	需要安装与插件兼容的 Blender 版本，并允许插件与脚本执行
插件运行环境	需要具备支撑插件逻辑的 Python 运行环境及相关依赖
数据存储环境	需要支持项目数据、日志数据和资源元数据的持久化存储
模型服务环境	需要支持本地或云端大语言模型接口访问
主系统界面	需要提供任务输入、参数配置、结果展示和后台治理能力

从部署角度看，软件环境要求的重点不只是“装上这些软件”，而是保证各组成部分之间能够稳定协同。例如，Blender 与插件版本之间应保持兼容，主系统与模型服务之间应具备清晰接口协议，项目数据和日志数据应具备可靠持久化能力。若上述环境之间版本不匹配或接口约定不一致，即使单个组件可独立运行，也难以形成完整系统。

(三) 运行方式要求

系统总体采用“主系统 + Blender 插件 + 数据与资源存储 + 模型服务”的协同运行方式。主系统负责面向用户的交互、权限控制、任务组织和结果展示；Blender 插件负责三维场景执行与对象生成；数据与资源存储负责项目数据、日志记录和资产索引；模型服务负责自然语言和多模态输入解析。四者之间通过明确的接口协同工作，共同完成场景生成任务。

从运行模式上看，系统更适合采用“主系统集中控制、插件按任务执行、模型服务按需调用、数据统一持久化”的协同方式，而不是让所有能力都混杂在同一个进程中。这样组织的好处在于：一方面，用户界面交互不会被底层插件执行细节完全绑死；另一方面，当模型服务、资源库或插件能力需要单独调整时，也更容易进行局部替换和维护。对于课程作业场景，这种描述能够较好体现系统架构关系，同时不会过早陷入具体实现细节。

(四) 部署与维护要求

为了保证系统在后续展示、复现实验结果和维护过程中具有可操作性，运行要求中还需要进一步明确部署与维护层面的基本要求。

1. 系统应支持独立部署主系统、模型服务和资源存储模块，以便后续根据运行环境灵活调整。
2. 系统应支持项目文件和日志文件的定期备份，避免因环境异常导致阶段性结果完全丢失。
3. 系统在插件版本、模型接口或资源库内容发生变化时，应能够通过配置调整完成适配，而不依赖整体重装。
4. 系统在展示或验收环境中，应能够提供基本的启动说明、运行说明和问题排查说明，以降低使用门槛。

(五) 外部接口要求

本文将外部接口分为用户接口、软件接口和通信接口三个层面进行说明。

1. 用户接口：系统应提供可供用户输入文本、上传草图、录入点线集、选择资源和查看结果的交互界面。
2. 软件接口：系统应提供主系统与 Blender 插件之间的任务接口，以及主系统与大语言模型服务之间的解析接口。
3. 通信接口：系统应支持主系统与模型服务、资源服务之间的网络通信，并保证状态和错误信息可回传。

(六) 运行要求补充说明

综合来看，系统运行要求的核心并不只是列出软硬件清单，而是确保用户交互层、插件执行层、模型解析层和数据存储层之间能够稳定协同。只有在运行环境、接口方式、部署要求和维护要求同时明确的情况下，系统需求才能从“纸面可描述”进一步走向“现实可运行”。因此，本章虽然放在文档后部，但其作用并不次于前文的功能需求和数据需求，而是对系统实际落地条件的集中说明。

八、 总结

本文围绕智能城市生成系统完成了较为详细的需求分析。与简单的功能罗列不同，本文在保持课程作业章节结构不变的前提下，尽量借鉴了软件需求规格说明文档的组织思想，将系统目标、用户特点、业务流程、数据对象、功能需求、质量约束和运行条件纳入统一框架之中。能够

在逻辑上回答系统为什么存在、由谁使用、做什么、怎么协同、受哪些限制以及应达到什么质量水平。

通过分析可以看出，智能城市生成系统的核心价值在于通过 Blender 插件执行、程序化建模、资源复用、多模态输入和大语言模型解析等能力的结合，帮助用户以更低的表达成本完成更高效率的三维城市场景构建与调整工作。与此同时，系统还通过动态仿真、权限治理和日志追踪等功能，提升了其在展示、分析和维护层面的完整性。本文档所形成的需求结果，可为后续的概要设计、数据库设计、接口设计和原型实现提供明确依据。